

Lab 3 - Propositional Logic Core Revision Summary

implication · conjunction · disjunction · equivalence · True/False · Lean proof-state tactics

◆ Basis: IO3.lean plus the worked explanations from this lab session

1. Topic Overview

This lab practices propositional proofs in Lean 4 by treating a proof as a sequence of transformations of the current proof state: assumptions appear above the turnstile and the remaining target appears after $\vdash$. The recurring skill is choosing a tactic from the shape of the goal or of an available assumption.

Lab logic: implication composition $\rightarrow$ argument swapping $\rightarrow$ build / unpack conjunctions $\rightarrow$ currying
equivalence $\rightarrow$ build / split disjunctions $\rightarrow$ True and ex-falso from False

2. Core Concepts and Tactics

Proof state and implication ($\rightarrow$)

- ▶ intro h: if the goal is $P \rightarrow Q$, assume P as $h : P$ and continue with goal Q .
- ▶ apply h: if $h : P \rightarrow Q$ and the current goal is Q , reduce the goal to P .
- ▶ assumption: close the goal when an assumption already has exactly the required type.
- ▶ exact term: explicitly provide a proof term whose type is exactly the current goal.
- ▶ $\rightarrow$ associates to the right: $P \rightarrow Q \rightarrow R$ means $P \rightarrow (Q \rightarrow R)$.

Conjunction ($\wedge$)

- ▶ Goal $P \wedge Q \rightarrow$ use constructor; Lean creates two goals: $\vdash P$ and $\vdash Q$.
- ▶ Assumption $pq : P \wedge Q \rightarrow$ use cases pq with $|$ intro $p\ q \Rightarrow \dots$ to unpack $p : P$ and $q : Q$.
- ▶ Direct projections are also available: $pq.1 : P$ and $pq.2 : Q$.
- ▶ Pair construction (p, q) has type $P \wedge Q$.

Disjunction ($\vee$)

- ▶ Goal $P \vee Q \rightarrow$ choose the branch with left or right, then prove that side.
- ▶ Assumption $pq : P \vee Q \rightarrow$ use cases pq ; prove the target separately for the $\text{inl } p$ and $\text{inr } q$ cases.

Equivalence ($\leftrightarrow$), True and False

- ▶ Goal $A \leftrightarrow B \rightarrow$ constructor creates the two directions $A \rightarrow B$ and $B \rightarrow A$.
- ▶ True is proved with constructor.
- ▶ From $f : \text{False}$, cases f proves any goal: this is ex falso quod libet.

3. Key Proof Patterns from the Lab

A. Implication composition — theorem C

```
theorem C : (P → Q) → ((Q → R) → (P → R)) := by
  intro pq
  intro qr
  intro p
  apply qr
  apply pq
  assumption
```

Reasoning: after the three intros, the state is $pq : P \rightarrow Q$, $qr : Q \rightarrow R$, $p : P$, $\vdash R$. Work backwards: to get R use qr , so prove Q ; to get Q use pq , so prove P ; P is already assumed.

B. Swap the order of inputs

```
theorem swap : (P → Q → R) → (Q → P → R) := by
  intro pqr q p
  apply pqr
  · assumption
  · assumption
```

Key point: `apply pqr` creates two subgoals, $\vdash P$ and $\vdash Q$. The bullet `·` scopes the tactics used for each subgoal.

C. Conjunction introduction and elimination

```
-- build P ∧ Q
example : P → Q → P ∧ Q := by
  intro p q
  constructor
  · assumption
  · assumption
```

```
-- unpack P ∧ Q
example : P ∧ Q → P := by
  intro pq
  cases pq with
  | intro p q => assumption
```

D. Curry / uncurry equivalence — the compact solution

```
theorem curry : (P → Q → R) ↔ (P ∧ Q → R) := by
  constructor
  · intro pqr pq
    cases pq with
    | intro p q =>
      exact pqr p q
  · intro pqr p q
    exact pqr (p, q)
```

Why this is smarter: `unpack P ∧ Q` once and feed p, q directly to `pqr`; in the reverse direction construct the conjunction directly as (p, q) . This avoids repeating `cases pq`.

E. Disjunction introduction and elimination

```
example : P → P ∨ Q := by
  intro p
  left
  assumption

example : (P → R) → (Q → R) → (P ∨ Q → R) := by
  intro pr qr pq
  cases pq with
  | inl p =>
    apply pr
    assumption
  | inr q =>
    apply qr
    assumption
```

4. Completed Practice Proofs

A. $(P \rightarrow R) \wedge (Q \rightarrow R) \leftrightarrow P \vee Q \rightarrow R$

```
example : (P → R) ∧ (Q → R) ↔ P ∨ Q → R := by
  constructor
  · intro prqr pq
    cases prqr with
    | intro pr qr =>
      cases pq with
      | inl p =>
        apply pr
        assumption
      | inr q =>
        apply qr
        assumption
  · intro pqr
    constructor
  · intro p
    apply pqr
    left
    assumption
  · intro q
    apply pqr
    right
    assumption
```

B. $P \wedge (Q \vee R) \leftrightarrow (P \wedge Q) \vee (P \wedge R)$ — distributivity

```
example : (P ∧ (Q ∨ R)) ↔ (P ∧ Q) ∨ (P ∧ R) := by
  constructor
  · intro pqr
    cases pqr with
    | intro p qr =>
      cases qr with
      | inl q =>
        left
        constructor
        · assumption
        · assumption
      | inr r =>
        right
        constructor
        · assumption
        · assumption
  · intro pqpr
    cases pqpr with
    | inl pq =>
      cases pq with
      | intro p q =>
        constructor
        · assumption
        · left
        assumption
    | inr pr =>
      cases pr with
      | intro p r =>
        constructor
        · assumption
        · right
        assumption
```

5. Tactic Decision Table

Situation	First move	What it does
Goal: $P \rightarrow Q$	intro p	Adds $p : P$; new goal Q
Have $h : P \rightarrow Q$; goal Q	apply h	Changes goal to P
Goal already appears in context	assumption	Closes the goal
Goal: $P \wedge Q$	constructor	Creates goals P and Q
Have $pq : P \wedge Q$	cases pq	Unpacks proofs of P and Q
Goal: $P \vee Q$	left / right	Choose which side to prove
Have $pq : P \vee Q$	cases pq	Handle P -case and Q -case separately
Goal: $A \leftrightarrow B$	constructor	Creates $A \rightarrow B$ and $B \rightarrow A$
Have $f : \text{False}$	cases f	No cases exist; any goal follows

6. Common Confusions / Revision Focus

- intro as a tactic and `| intro p q =>` inside cases look similar, but the latter names the constructor used to build an And proof.
- apply works backwards from the goal and may create more than one subgoal; bullets · separate those branches.
- For $\vee$: when it is the goal, choose left/right; when it is an assumption, split with cases.
- For $\wedge$: when it is the goal, use constructor; when it is an assumption, unpack with cases.
- For $\leftrightarrow$, think “two implications”; constructor is therefore the natural first step.
- Must be able to write from memory: C, swap, conjunction build/unpack, curry, disjunction case split, True, and False $\rightarrow$ P.